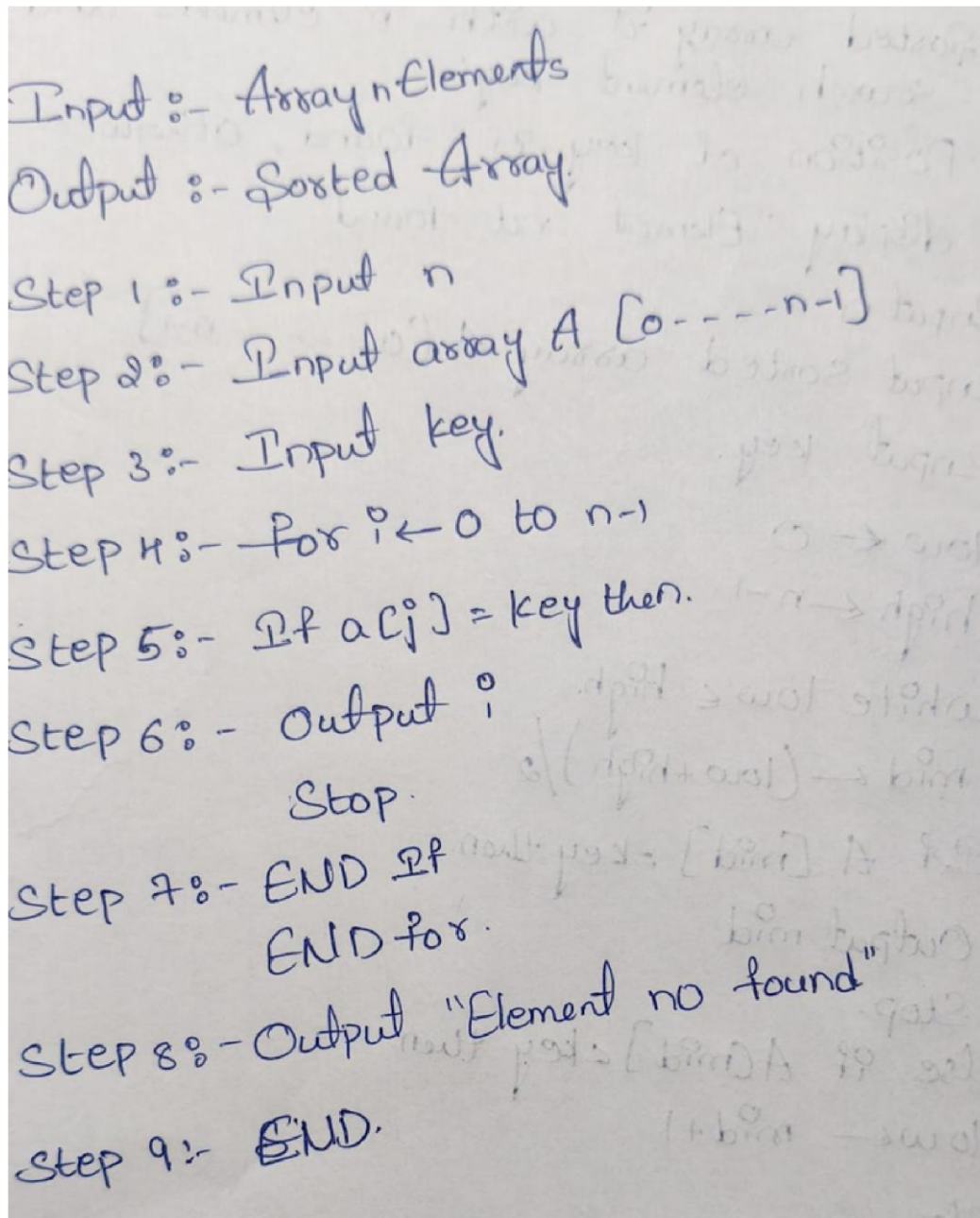


Practical-2

Aim:

Implementation and Time analysis of linear and binary search algorithm.

1.Linear search Algorithm:



Input :- Array n Elements
Output :- Sorted Array.
Step 1 :- Input n
Step 2 :- Input array A [0 ---- n-1]
Step 3 :- Input key.
Step 4 :- For $i \leftarrow 0$ to $n-1$
Step 5 :- If $a[i] = \text{key}$ then.
Step 6 :- Output i
 Stop.
Step 7 :- END If
 END for.
Step 8 :- Output "Element no found"
Step 9 :- END.

Program code:

```
#include <iostream>

using namespace std;

int linearsearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i; // return index if found
        }
    }
    return -1; // return -1 if not found
}

int main() {
    int arr[] = {2, 3, 4, 5, 6, 7, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 8;
    int result = linearsearch(arr, n, key);

    if (result == -1) {
        cout << "Element not found";
    } else {
        cout << "The element found at index " << result;
    }

    return 0;
}
```

Output:

The element found at index 6

Time complexity:

① Best case:-
The element is present at the first position
Ex:- $A = 10 \ 20 \ 30 \ 40 \ 50$
key = 10
 $T(n) = 1$
 $T(n) = O(1)$

② Average case:-
Suppose the element is somewhere in the middle
Ex:- $A = 10 \ 20 \ 30 \ 40 \ 50$
key = 30
 $T(n) = n/2$
 $T(n) = O(n)$

③ Worst case:-
Element in last position (or) Not element present
 $A = 10 \ 20 \ 30 \ 40 \ 50$
key = 50
 $T(n) = n$
 $T(n) = O(n)$

Conclusion:

Linear search searches the array by comparing each element with the key one by one. It returns the position of the key if it is found; otherwise, it reports that the element is not found.

2. Binary search **Algorithm:**

Input :- Sorted array 'A' with 'n' elements. and
search element 'key'.
Output :- Position of key if found, otherwise
display "Element not found".

Step 1:- Input n
Input sorted array A[0, ---- n-1]
Input key.

Step 2:- $low \leftarrow 0$
 $high \leftarrow n-1$

Step 3:- while $low \leq high$.
 $mid \leftarrow (low + high) / 2$

Step 4:- If $A[mid] = key$ then.
Output mid
Stop.

Step 5:- else if $A[mid] < key$ then.
 $low \leftarrow mid + 1$

Step 6:- Else
 $high \leftarrow mid - 1$

Step 7:- END If
END while.

Step 8:- Output "Element not found"

Step 9:- END.

Program:

```
#include <iostream>

using namespace std;

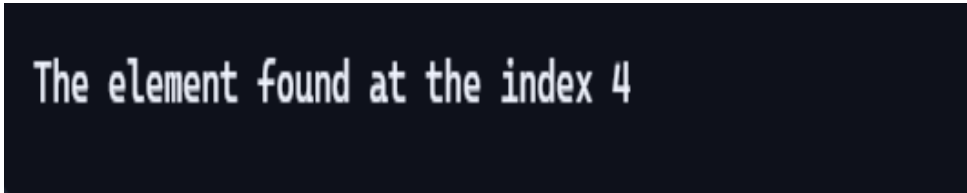
int binarysearch(int arr[], int size, int key) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) {
            return mid; // found
        }
        else if (arr[mid] < key) {
            low = mid + 1; // search right half
        }
        else {
            high = mid - 1; // search left half
        }
    }
    return -1; // not found
}
```

```
int main() {  
    int arr[] = {1, 2, 3, 4, 5, 6};  
    int size = sizeof(arr) / sizeof(arr[0]);  
    int key = 5;  
  
    int result = binarysearch(arr, size, key);  
  
    if (result == -1) {  
        cout << "The element not found";  
    } else {  
        cout << "The element found at the index " << result;  
    }  
  
    return 0;  
}
```

Output:



```
The element found at the index 4
```


Time complexity:

Best Case:-
The element is found at the middle position in the first comparison.

Ex:- $A = 10 \ 20 \ 30 \ 40 \ 50$
key = 30
 $A[\text{mid}] = \text{key}$
 $T(n) = 1$
 $T(n) = O(1)$

Average Case:-
On Average the search also eliminates approximately half of the remaining element at each step.

$T(n) \approx \log_2 n$
 $T(n) = O(\log n)$

Worst Case:-
This is where Binary Search becomes interesting

$n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$

$n/2^k = 1$
 $n = 2^k$
 $k = \log_2 n$
 $T(n) = \log n$
 $T(n) = O(\log n)$

Conclusion:

Binary search searches the sorted array by repeatedly dividing the search range into two halves. It efficiently finds the key element and returns its position if found.